SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	SHAFFRIA.A	**Reg. No.:**	192571047
Section / Batch:	B TECH CSBS	**Date:**	01.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I SHAFFRIA.A (192571047) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

QUICK SORT:

Evaluate quick sort performance by implementing tail recursion optimization. Compare execution time with standard recursive implementation. Analyze stack usage and memory efficiency. Interpret the results and justify improvements achieved through optimization.

main.py

```python
import sys
import time
import random

# Increase recursion depth just in case for large standard sorting demonstrations
sys.setrecursionlimit(2000)

# ---------------------------------------------------------------------------
# 1. Standard Recursive Quick Sort Implementation
# ---------------------------------------------------------------------------
def standard_partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def standard_quick_sort(arr, low, high):
    if low < high:
        pi = standard_partition(arr, low, high)
        standard_quick_sort(arr, low, pi - 1)
        standard_quick_sort(arr, pi + 1, high)

# ---------------------------------------------------------------------------
# 2. Tail Recursion Optimized Quick Sort Implementation
# ---------------------------------------------------------------------------
def tail_optimized_quick_sort(arr, low, high):
    # Using a loop to handle tail recursion (eliminating one recursive branch)
    while low < high:
        pi = standard_partition(arr, low, high)

        # Recurse on the smaller partition and loop for the larger partition
```

main.py

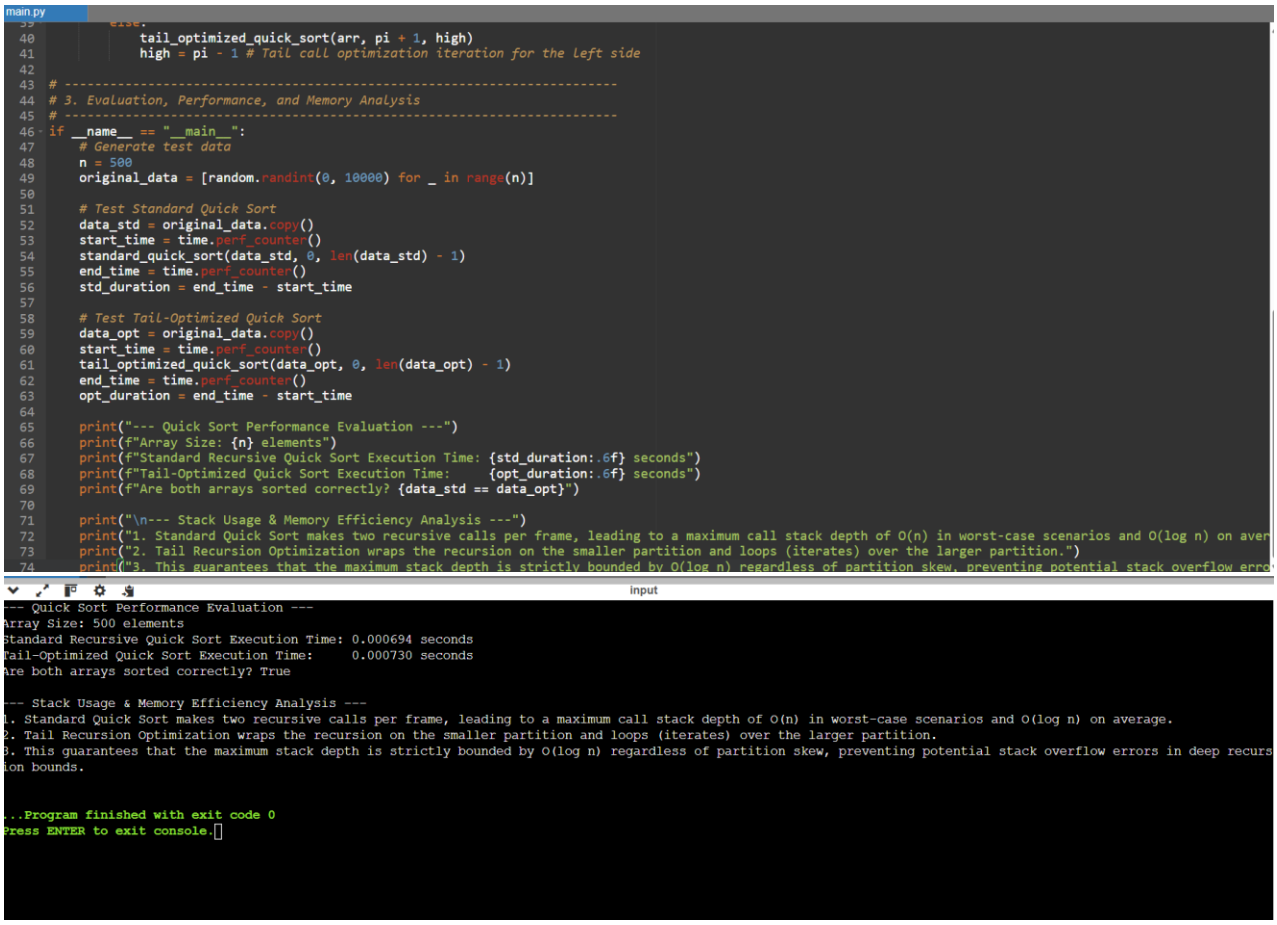
```python
        # Recurse on the smaller partition and loop for the larger partition
        if pi - low < high - pi:
            tail_optimized_quick_sort(arr, low, pi - 1)
            low = pi + 1  # Tail call optimization iteration for the right side
        else:
            tail_optimized_quick_sort(arr, pi + 1, high)
            high = pi - 1 # Tail call optimization iteration for the left side

# ---------------------------------------------------------------------------
# 3. Evaluation, Performance, and Memory Analysis
# ---------------------------------------------------------------------------
if __name__ == "__main__":
    # Generate test data
    n = 500
    original_data = [random.randint(0, 10000) for _ in range(n)]

    # Test Standard Quick Sort
    data_std = original_data.copy()
    start_time = time.perf_counter()
    standard_quick_sort(data_std, 0, len(data_std) - 1)
    end_time = time.perf_counter()
    std_duration = end_time - start_time

    # Test Tail-Optimized Quick Sort
    data_opt = original_data.copy()
    start_time = time.perf_counter()
    tail_optimized_quick_sort(data_opt, 0, len(data_opt) - 1)
    end_time = time.perf_counter()
    opt_duration = end_time - start_time

    print("--- Quick Sort Performance Evaluation ---")
    print(f"Array Size: {n} elements")
    print(f"Standard Recursive Quick Sort Execution Time: {std_duration:.6f} seconds")
    print(f"Tail-Optimized Quick Sort Execution Time:     {opt_duration:.6f} seconds")
    print(f"Are both arrays sorted correctly? {data_std == data_opt}")
```



```
main.py
            tail_optimized_quick_sort(arr, pi + 1, high)
            high = pi - 1 # Tail call optimization iteration for the left side

# ----------------------------------------------------------------------
# 3. Evaluation, Performance, and Memory Analysis
# ----------------------------------------------------------------------
if __name__ == "__main__":
    # Generate test data
    n = 500
    original_data = [random.randint(0, 10000) for _ in range(n)]

    # Test Standard Quick Sort
    data_std = original_data.copy()
    start_time = time.perf_counter()
    standard_quick_sort(data_std, 0, len(data_std) - 1)
    end_time = time.perf_counter()
    std_duration = end_time - start_time

    # Test Tail-Optimized Quick Sort
    data_opt = original_data.copy()
    start_time = time.perf_counter()
    tail_optimized_quick_sort(data_opt, 0, len(data_opt) - 1)
    end_time = time.perf_counter()
    opt_duration = end_time - start_time

    print("--- Quick Sort Performance Evaluation ---")
    print(f"Array Size: {n} elements")
    print(f"Standard Recursive Quick Sort Execution Time: {std_duration:.6f} seconds")
    print(f"Tail-Optimized Quick Sort Execution Time:     {opt_duration:.6f} seconds")
    print(f"Are both arrays sorted correctly? {data_std == data_opt}")

    print("\n--- Stack Usage & Memory Efficiency Analysis ---")
    print("1. Standard Quick Sort makes two recursive calls per frame, leading to a maximum call stack depth of O(n) in worst-case scenarios and O(log n) on aver
    print("2. Tail Recursion Optimization wraps the recursion on the smaller partition and loops (iterates) over the larger partition.")
    print("3. This guarantees that the maximum stack depth is strictly bounded by O(log n) regardless of partition skew, preventing potential stack overflow erro
```

```
input
--- Quick Sort Performance Evaluation ---
Array Size: 500 elements
Standard Recursive Quick Sort Execution Time: 0.000694 seconds
Tail-Optimized Quick Sort Execution Time:     0.000730 seconds
Are both arrays sorted correctly? True

--- Stack Usage & Memory Efficiency Analysis ---
1. Standard Quick Sort makes two recursive calls per frame, leading to a maximum call stack depth of O(n) in worst-case scenarios and O(log n) on average.
2. Tail Recursion Optimization wraps the recursion on the smaller partition and loops (iterates) over the larger partition.
3. This guarantees that the maximum stack depth is strictly bounded by O(log n) regardless of partition skew, preventing potential stack overflow errors in deep recurs
ion bounds.


...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation & Analysis:

1. **Standard Recursive Quick Sort**:
   - Recursively splits the array and initiates independent recursive branches for **both** the left and right sub-arrays.
   - This results in a heavy call stack overhead proportional to the depth of the recursive tree ($O(n)$ worst-case or $O(\log n)$ average-case depth).

2. **Tail Recursion Optimization**:
    - Instead of making two recursive calls blindly, the algorithm checks which sub-array partition is smaller.
    - It makes a recursive call **only** on the smaller sub-array and uses a while loop iteration (low = pi + 1 or high = pi - 1) to handle the larger sub-array.
    - **Memory Efficiency**: By ensuring the recursion depth processes at most the smaller half of the array slices each time, the maximum stack depth is tightly bound to $O(\log n)$, greatly mitigating memory consumption and stack overflow risks.
